Автономная ферма для решения task-based CTF

Рой агентов на DeepSeek-V4 в режиме оплаты по API. Инфраструктура координации, логика конвейера и оптимизация потоков агентов.

Режим: только API · без self-host

Дата: 20 сентября 2026 Модель-воркер: DeepSeek V4-Flash · эскалация: V4-Pro Обновление к v1 (37 стр.)

— Содержание

1. Резюме и главный тезис
2. Решение по хостингу: только API
3. Экономика токенов: сколько стоит «прогон»
4. Каскад моделей: Flash + эскалация в Pro
5. Архитектура: пять слоёв
6. Логика работы: сквозной конвейер
7. Координация роя и оптимизация ресурсов
8. Специализация агентов и тулкиты
9. Песочница и безопасность исполнения
10. Контур накопления опыта (в API-режиме)
11. Референс-стек
12. Дорожная карта
13. Реалистичные ожидания
14. Риски и правила соревнований

1 Резюме и главный тезис

Ферма — это управляемый рой недорогих агентов V4-Flash, которые в изолированных песочницах параллельно и разными стратегиями атакуют задачи CTF, а централизованный планировщик распределяет между ними бюджет и подсказки. Дорогая модель V4-Pro подключается редко и точечно — только когда дешёвый агент «застрял».

Ключевой вывод, подтверждённый опытом DARPA AIxCC 2025 (семь автономных систем сутками сами искали и эксплуатировали уязвимости): **выигрыш даёт не «модель помощнее» и не «больше агентов», а инженерная надёжность и координация**. Победитель выиграл за счёт стабильности и ансамблевой избыточности, а несколько технически более изощрённых команд сошли с дистанции из-за крашей и утечек ресурсов. Поэтому 60–70% ценности фермы — это оркестрация, изоляция, дедупликация и управление бюджетом, и лишь остаток — «интеллект» самой модели.

> **Три опоры фермы.** (1) **Каскад моделей** — рой Flash + редкая эскалация в Pro даёт качество Pro-класса при цене, близкой к Flash. (2) **Качество инструментов и песочницы** — живой доступ к `gdb`, сокету, дизассемблеру важнее апгрейда модели (урок EnIGMA). (3) **Контур накопления опыта** — банк решений (RAG), который делает рой умнее с каждым турниром.

~$0.12	~15%	cost / flag
ЗА ТРАЕКТОРИЮ (FLASH)	ДОЛЯ ЭСКАЛАЦИЙ В PRO	ГЛАВНАЯ МЕТРИКА

2 Решение по хостингу: только API

Зафиксированное решение: **никакого выделенного сервера под self-hosted V4-Flash**. Модель используется через API DeepSeek, оплата — по факту за токены. Это правильный выбор для данного масштаба, и вот почему.

Self-host окупается только на очень большом устойчивом потоке (десятки тысяч траекторий в сутки), при требовании секретности данных или при потребности в собственном дообучении на открытых весах. На малом→среднем масштабе он даёт лишь минусы: V4-Pro (1.6T параметров) — это мульти-узловой инференс на дорогих GPU, а даже V4-Flash (284B) требует multi-GPU и постоянного DevOps-сопровождения (батчинг, автоскейл, KV-кэш под 1M-контекст). Против этого API даёт мгновенную эластичность «включил → заплатил центы → выключил».

Единственный компонент, который мы всё равно строим сами, — это **тонкий модельный шлюз** (LiteLLM-класс): он маршрутизирует запросы между Flash и Pro, ведёт учёт стоимости по каждой траектории, делает фолбэк при ошибках провайдера и страхует от вендор-лока (в шлюз можно подмешать второго провайдера тех же весов). Абстракция стоит дёшево, а свободу манёвра сохраняет.

Критерий	API (наш выбор)	Self-host
Порог входа	Ключ и HTTP-клиент	GPU-узлы, инференс-стек, DevOps
Стоимость на малом объёме	Центы за траекторию	Аренда GPU 24/7 вне зависимости от загрузки
Эластичность	Мгновенная, до нуля	Ограничена парком железа
Prompt-кэш	Автоматический, на стороне провайдера	Нужно поднимать самому (RadixAttention)
Дообучение на весах	Недоступно (только provider-side, если есть)	Полный контроль (DPO/RFT)

Честная оговорка про self-host. Отказ от него означает, что «ров» в виде дообучения собственной модели на накопленных траекториях в этой версии недоступен. Взамен мы выжимаем максимум из того, что даёт API: банк решений через RAG, few-shot-демонстрации и, если DeepSeek предоставляет provider-side fine-tuning, — точечное дообучение через их сервис (см. раздел 10).

3 Экономика токенов: сколько стоит «прогон»

Актуальные тарифы (проверено 18.09.2026; источники по Pro расходятся, перед «боем» сверить). Ключевое — **разница между Flash и Pro почти на порядок**, и это главный рычаг экономики.

Модель	Вход (miss)	Вход (cache-hit)	Выход	Контекст
V4-Flash (~284B)	$0.14 / 1M	$0.0028 / 1M	$0.28 / 1M	~1M
V4-Pro (~1.6T)	$0.435 / 1M	$0.022 / 1M	$0.87 / 1M	~1M

Одна попытка решения задачи (траектория) — это агентный цикл на 30–80 шагов, где контекст растёт с каждым вызовом инструмента. Но большая часть входных токенов — это повторяющийся префикс (системная инструкция, описание задачи, дамп файлов), который DeepSeek кэширует автоматически и биллит в ~50 раз дешевле. Отсюда — грубая, но честная смета на Flash:

```
траектория Flash ≈ вход 2.5M токенов (80% cache-hit)
  miss: 0.5M × $0.14/1M = $0.070
  hit: 2.0M × $0.0028/1M = $0.006
  выход: 0.15M × $0.28/1M = $0.042
──────────────────────────────────
итого ≈ $0.12 за траекторию
```

Дальше это масштабируется предсказуемо. Ансамбль из ~6 траекторий на задачу ≈ **$0.7**. Эскалация в Pro примерно на 15% задач добавляет в среднем ~$0.15/задачу. Средняя стоимость решённой задачи — порядка **$0.85**.

Сценарий прогона	Задач в работе	Ансамбль	Порядок стоимости
Лёгкий («пара долларов»)	3–5	2–3	$1–3
Рабочий	10–15	4–6	$8–15
Агрессивный грайнд турнира	25+	8–12	$30–60

> **Управление ожиданиями.** «Запустил, заплатил пару долларов — и всё» — это реалистично для *лёгкого* прогона (несколько задач или жёсткий потолок бюджета). Полноценный конкурентный грайнд с широкими ансамблями — это уже десятки долларов за турнир. Хорошая новость: стоимость линейна и полностью под контролем губернатора бюджета (раздел 7), поэтому «пару долларов» можно задать как жёсткий лимит и рой просто остановится, упёршись в него.

4 Каскад моделей: Flash + эскалация в Pro

Гомогенный рой из Pro был бы качественным, но дорогим; гомогенный рой из Flash — дешёвым, но с проседанием на сложных задачах (hard-pwn, продвинутая крипта). Решение — **каскад**: по умолчанию всё делают дешёвые Flash-воркеры, а Pro подключается как «тяжёлая артиллерия» только по триггеру.

Триггеры эскалации

- **Застревание.** Нет прогресса за N шагов (не растёт покрытие, повторяются одни и те же действия) — поднимаем траекторию на Pro с сохранённым контекстом.
- **Высокая ставка.** Задача дорогая по очкам и/или горит дедлайн — Pro подключается сразу к части ансамбля.
- **Верификация гипотезы.** Flash нашёл правдоподобный вектор, но не может довести эксплойт — Pro «дожимает» конкретный шаг.

При доле эскалаций ~15% каскад даёт качество, близкое к Pro-рою, при цене, близкой к Flash-рою — экономия порядка **4×** против гомогенного Pro. Этот приём напрямую перенесён из AIxCC (команда Lacrosse эскалировала «дешёвая→дорогая», а Buttercup сознательно избегала дорогих reasoning-моделей там, где хватало дешёвых).

> Важно: и Flash, и Pro в нашей схеме — это **API-вызовы через один шлюз**. Эскалация — это не переключение железа, а смена `model` в запросе с переносом текущего контекста траектории. Это делает лестницу эскалации почти бесплатной в реализации.

5 Архитектура: пять слоёв

Ферма разложена на пять слоёв. Принцип: «интеллект» (агент + LLM) — тонкий и заменяемый, а всё тяжёлое и критичное к надёжности — детерминированное.



Слои фермы. Через слои 1–5 проходит каждая попытка; LLM-доступ вынесен в единый шлюз.

1 · Control plane

Центральный планировщик принимает задачи, оценивает их ожидаемую ценность (очки / оценочная стоимость решения), кладёт в приоритетную очередь и раздаёт воркерам. Поверх — **durable-оркестратор** (Temporal или Restate): попытка решения — это долгий, падающий процесс с состоянием, и краш воркера не должен терять прогресс или «подвешивать» задачу. Именно на потере состояния теряли очки в AIxCC. GPU/сетевые операции идут в activities с heartbeat меньше половины таймаута.

2 · Execution plane — сердце CTF-фермы

Каждая попытка живёт в эфемерном изолированном контейнере со своим тулчейном и контролируемой сетью. На изоляции нельзя экономить: агент запускает недоверенный код и эксплойты. Практично — **Firecracker/gVisor microVM на попытку**, сеть по умолчанию deny с явным egress только на целевой эндпоинт задачи, лимиты cgroups, watchdog на зависания, и никаких секретов внутри песочницы.

3 · Agent runtime

Не переусложнять фреймворком. Базовый цикл — ReAct/tool-loop: наблюдение → гипотеза → действие в песочнице → результат → уточнение. LangGraph или совместимый Agents-SDK годятся как каркас с чекпойнтами, но сильный CTF-агент — это часто несколько сот строк вокруг цикла «tool-calling + жёсткий шаблон под категорию».

4 · Shared memory — то, что делает рой «роем»

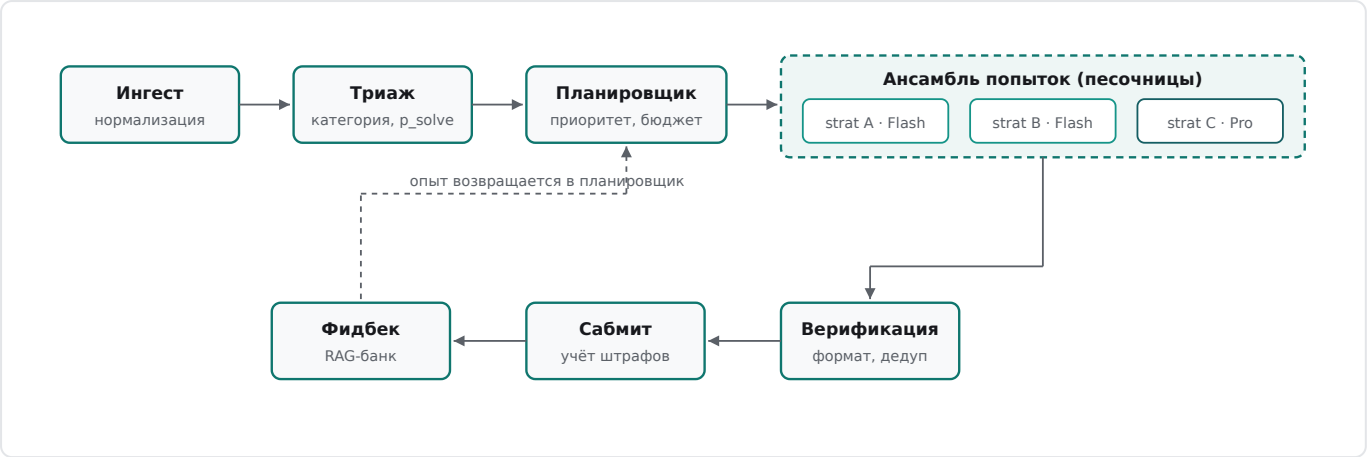
Общий слой знаний: (а) банк прошлых решений и writeup-ов, (б) библиотека шаблонов эксплойтов и приёмов по категориям, (в) «доска» (blackboard) для обмена промежуточными находками между параллельными попытками одной задачи. Это прямое воплощение ансамблевого урока AIxCC — модули делятся «семенами» и подсказками.

5 · Observability и cost governance

Трейсинг каждой попытки (Langfuse-класс), жёсткие бюджеты (лимит токенов / времени / итераций на задачу), kill-switch-и, дашборд «стоимость за флаг». Без этого рой либо разоряется на зацикленных попытках, либо тихо деградирует. Трейсинг включаем с первого дня — он же копит датасет на будущее.

6 Логика работы: сквозной конвейер

Задача проходит стадии, и на каждой есть детерминированная проверка перед передачей дальше (staged validation — ещё один урок AIxCC: несколько слоёв проверки спасают от ошибочных сабмитов и штрафов).



Сквозной конвейер: от приёма задачи до возврата опыта в банк решений.

Ингест и нормализация. Источники (архивы прошлых CTF, тренировочные платформы, живое соревнование через API/CTFd) приводятся к единому объекту задачи: файлы, описание, категория, remote-эндпоинт, формат флага, стоимость в очках, дедлайн.

Триаж. Быстрый дешёвый проход (Flash или эвристики): определить категорию, оценить решаемость `p_solve` и сложность, отсеять заведомо неподъёмное. Отсюда — приоритет в очереди.

Планирование и раздача. Координатор направляет задачу в специализированный пул агентов по категории и запускает **не одну, а несколько параллельных попыток разными стратегиями** (разные температуры, углы атаки, а по триггеру — модель Pro). Первая попытка, добывшая валидный флаг, «выигрывает», остальные родственные отменяются, ресурсы освобождаются.

Solve-цикл. Внутри песочницы агент крутит ReAct с инструментами своей категории, итеративно строит и запускает эксплойт, наблюдает вывод, уточняет. Здесь работает лестница эскалации: начали на Flash, застряли — подняли на Pro.

Верификация и сабмит. Проверка формата флага, дедупликация (не сабмитить одно дважды), сам сабмит, учёт штрафов за неверные попытки — детерминированно, без LLM там, где можно.

Фидбек. Успешные решения → в банк как шаблоны; неудачи логируются с причинами; со временем формируется «учебный план» — где рой слаб, туда точечно докидываются шаблоны и демонстрации.

7 Координация роя и оптимизация ресурсов

Это ядро вопроса «как рой эффективно координирует ресурсы». Собранные вместе, эти механизмы превращают «пачку агентов» в эффективный рой.

Планировщик как задача о рюкзаке

Свободный слот всегда тратится на задачу с лучшим отношением ценности к стоимости, а не на первую попавшуюся. Формула приоритета:

```
приоритет ≈ (points × p_solve × decay) / est_cost + deadline_boost − sink_penalty
```

где `decay` — падение стоимости задачи при динамическом скоринге, `deadline_boost` поднимает горящие задачи, а `sink_penalty` штрафует «чёрные дыры», куда уже влит бюджет без прогресса. Цель — максимизировать ожидаемые очки к дедлайну.

Тройной бюджет на траекторию

Жёсткие потолки одновременно по **шагам**, **токенам** и **времени**. Как только траектория перестаёт давать прогресс — стоп, а не бесконечный цикл (главный источник слитого бюджета у агентов). Глобальный дневной бюджет фермы задаётся сверху — то самое «пару долларов» как жёсткий лимит.

Ансамбль и ранняя отмена

N разнообразных параллельных попыток, first-to-flag отменяет остальные. Разнообразие важнее количества: разные стратегии ловят разные баги (у победителя AIxCC — 8 патчеров с разными стратегиями, у FuzzingBrain — 23 независимые).

Blackboard-обмен между попытками

Параллельные агенты по одной задаче делятся находками (нашли смещение оффсета, поняли схему шифрования, вычислили canary) — это резко срезает дублирование. Прямой аналог «shared-memory seed sharing» из AIxCC. Координация — менеджер-воркеры + work-stealing + contract-net для перераспределения.

Эксплуатация prompt-кэша

Промпты строятся так, чтобы длинный стабильный префикс (системная инструкция + дамп файлов задачи) шёл первым и переиспользовался всеми попытками одной задачи — тогда большая часть входа биллится по cache-hit ($0.0028 вместо $0.14). На рое это экономит десятки процентов бюджета почти бесплатно и особенно ценно в API-режиме, где кэш работает автоматически.

Backpressure и лимиты конкурентности

Ограничение одновременных песочниц и RPS к API, чтобы не упереться в rate-limit провайдера и не устроить себе DoS. Оркестратор держит очередь и добирает работу по мере освобождения слотов; при дефиците включается admission control — часть задач не берётся в работу вовсе.

8 Специализация агентов и тулкиты

Одна общая модель (Flash), но разные промпты, наборы инструментов и демонстрации под каждую категорию — это и даёт специализацию, и сохраняет общий prefix-кэш.
Категорийные пулы:

Категория	Ключевые инструменты	Сила ИИ
pwn / rev	pwntools, GDB+pwndbg, radare2/rizin, Ghidra (headless), angr, ROPgadget, one_gadget	Средняя; hard-pwn — слабое место
crypto	SageMath, PyCryptodome, z3, шаблоны (RSA-типовые, LLL/lattice, padding-oracle)	Шаблонное — хорошо; продвинутое — слабо
web	headless-браузер, перехват-прокси, ffuf, sqlmap-класс, набор пейлоадов	Сильная сторона ИИ
forensics / misc	binwalk, foremost, volatility, стего-инструменты, exiftool	Сильная сторона ИИ

Триаж направляет задачу в нужный пул; при неоднозначной категории задача может уйти сразу в два пула как часть ансамбля. Демонстрации (few-shot) для каждой категории берутся из банка решений — это дешёвый способ поднять качество без дообучения.

9 Песочница и безопасность исполнения

Execution plane — самая критичная к надёжности и безопасности часть, потому что мы намеренно запускаем недоверенный код в масштабе.

- **Эфемерность.** Свежая microVM на каждую попытку, уничтожается после завершения — никакого переноса состояния между задачами.
- **Сетевой allowlist.** Egress по умолчанию запрещён, открывается только на целевой эндпоинт конкретной задачи. Это же защищает от того, чтобы агент случайно «вышел» за пределы санкционированной цели.
- **Лимиты ресурсов.** cgroups на CPU/RAM/диск, watchdog на зависшие процессы, таймауты на каждый инструмент.
- **Без секретов внутри.** Ключи API и токены сабмита живут в control plane, а не в песочнице; агент получает только то, что нужно для конкретного шага.

> **Область применения — строго санкционированные цели.** Ферма работает по тренировочным архивам, собственным инстансам и соревнованиям, где автоматизация разрешена правилами. Автономная генерация эксплойтов против чужих систем — это уже не CTF; такие цели в ферму не подаются, и сетевой allowlist это технически ограничивает.

10 Контур накопления опыта в API-режиме

Без self-host полноценное дообучение на открытых весах (DPO/RFT) недоступно. Но контур самоулучшения всё равно строится — просто другими средствами, доступными поверх API:

- **Банк решений (RAG).** Каждое успешное решение раскладывается на переиспользуемые фрагменты: вектор атаки, ключевой инструмент, шаблон эксплойта. При новой задаче похожие фрагменты подмешиваются в контекст. Это главный, полностью доступный в API-режиме источник роста.
- **Оптимизация промптов и few-shot.** Демонстрации под категории обновляются по мере накопления удачных траекторий; слабые места закрываются точечными примерами.
- **Provider-side fine-tuning.** Если DeepSeek предоставляет дообучение через свой сервис — это опция получить «свой» вес без своего железа. Требует проверки доступности и условий (см. ниже).
- **Курируемый датасет.** Трейсинг с первого дня копит структурированный корпус траекторий — он ценен и как обучающая выборка, и как аналитика «где рой слаб».

> Честный компромисс: отказавшись от self-host, мы меняем самый сильный, но самый дорогой «ров» (собственная дообученная модель) на набор дешёвых и быстрых приёмов. Для малого→среднего масштаба это правильный размен; вернуться к дообучению можно позже, если объём вырастет.

11 Референс-стек

Слой	Выбор	Зачем
Модель-воркер	DeepSeek V4-Flash (API)	Дёшево, быстро, авто-кэш префиксов
Эскалация	DeepSeek V4-Pro (API)	Точечно на застрявших/дорогих задачах
Модельный шлюз	LiteLLM-класс	Роутинг Flash/Pro, учёт стоимости, фолбэк, анти-vendor-lock
Оркестрация	Temporal (или Restate)	Durable-workflow, ретраи без потери прогресса
Очередь	Redis Streams → NATS при росте	Раздача задач, backpressure
Координация	Blackboard (Postgres + Redis)	Обмен находками, work-stealing, contract-net
Песочница	Firecracker / gVisor, эфемерные	Изоляция недоверенного кода, egress-allowlist
Агент	Тонкий ReAct на LangGraph-класс	Чекпойнты, tool-calling, шаблоны по категориям
Данные	Postgres + Redis + MinIO + pgvector	Состояние, кэш, артефакты, RAG-банк
Наблюдаемость	Langfuse-класс + OpenTelemetry	Трейсинг, метрики, датасет на будущее

Главные метрики (KPI): `cost-per-flag` (стоимость за решённый флаг), `time-to-flag` и `escalation-rate` (доля обращений к Pro). По ним же принимаются решения о переходе между фазами.

12 Дорожная карта

Разворачивать поэтапно, переходя по метрике, а не по календарю. Это бьёт в главный урок AIxCC: система, которая надёжно применяет базовые техники, уже была бы в топ-3 — инженерная зрелость важнее алгоритмической новизны.

Фаза	Что делаем	Критерий перехода
Ф0 · MVP	1 агент, 1 категория (crypto — детерминированные тулы), песочница, лимит бюджета, API	Стабильно решает известный корпус
Ф1 · Рой на API	Очередь + пул воркеров, blackboard, триаж, каскад Flash→Pro	Ансамбль обгоняет одиночного агента по cost-per-flag
Ф2 · Durable	Temporal-воркфлоу, эфемерные песочницы на попытку, backpressure	Нет потерь прогресса при крашах воркеров
Ф3 · Live-планировщик	Приоритет-knapsack, тройной бюджет, admission control, «дожатие» под дедлайн	Держит целевой бюджет на реальном турнире
Ф4 · Самоулучшение	RAG-банк решений, few-shot по категориям, (опц.) provider-side fine-tuning	Метрики растут от турнира к турниру

Трейсинг — с первого дня: копить датасет и аналитику слабых мест ещё до того, как понадобится контур обучения.

13 Реалистичные ожидания

Что показывают бенчмарки 2024–2026, чтобы не строить иллюзий:

Ориентир	Результат
InterCode-CTF (лёгкие задачи)	до ~95% (ReAct-агенты)
NYU CTF Bench (реальные задачи)	~13.5% (EnIGMA); хвост hard всё ещё низкий
CAI на Neurogrid CTF (2025)	41/45 флагов, победа над людьми
HackTheBox после LLM	медианное время: easy −47%, medium −32%, hard −27%

Вывод: **ферма отлично выгребает лёгкие и шаблонные задачи** (web, forensics, типовые шаги) и заметно ускоряет средние, но **сложный хвост** (hard-pwn, продвинутая крипта) остаётся преимущественно за людьми. Стратегия «много дешёвых решённых задач + точечная эскалация на дорогих» ровно под это и заточена.

14 Риски и правила соревнований

- **Правила турниров.** Часть CTF запрещает автоматизацию или выделяет отдельные AI-дивизионы. Читать ToS до участия — это вопрос допуска, а не только этики.
- **Тариф V4-Pro плавает.** Источники расходятся ($0.4–1.7 за 1M входа); перед «боем» сверять актуальную официальную цену и лимиты API.
- **Runaway-затраты.** Без жёстких бюджетов и watchdog зациклившийся рой способен «сжечь» деньги — губернатор бюджета обязателен с Ф0.
- **Rate-limit провайдера.** В API-режиме масштаб роя ограничен лимитами DeepSeek; backpressure и, при необходимости, второй провайдер в шлюзе.
- **Область применения.** Только санкционированные/тренировочные цели; egress-allowlist делает это техническим ограничением, а не только договорённостью.

Что перепроверить перед запуском

- Актуальный официальный тариф и rate-limit V4-Flash / V4-Pro API.
- Наличие и условия provider-side fine-tuning у DeepSeek (для Ф4).
- Правила конкретных соревнований, где планируется участие.

Итог. В API-режиме ферма — это дешёвый в старте и полностью эластичный рой: тонкие агенты V4-Flash под управлением надёжного планировщика, каскадом дотягивающиеся до V4-Pro на трудных задачах, в изолированных песочницах, с банком решений вместо собственной дообученной модели. «Пару долларов» — это реальный потолок для лёгкого прогона, а не маркетинг; масштаб и стоимость растут линейно и остаются под жёстким контролем бюджета.

Проект slopi-farm · Технический блюпринт v2 (API-режим) · 20.09.2026. Обновляет v1 (37 стр.) в части хостинга: self-host и гибрид сняты в пользу чистого API.